

8h às 10h

Assista aos vídeos e leia os artigos indicados antes do encontro.

Autoestudo:

1

Deploy de modelo e pipeline de processamento: o tema central do encontro de hoje

3

MLflow: primeiros passos — rastreamento de experimentos (documentação oficial, em inglês)

2

Como exportar um modelo preditivo: salvando com pickle e joblib (Machine Learning Mastery)

4

Como usar Pipelines no Scikit-Learn: unindo pré-processamento e modelo em um só objeto (Data Hackers)

| Stand-up daily: em pé, direto ao ponto

15 min

10:00 → 10:15

1

Em pé

A daily é rápida — todos de pé.

2

Tasks do projeto

O que cada um fez e o que falta.

3

Cronograma do dia

Definir prioridades e metas de hoje.

Objetivo: alinhar o time e destravar o dia.

DIAGRAMA DO MÓDULO



Deploy de Modelo e Criação de Pipeline de Processamento

Exportação de Modelos • Pipelines no Scikit-Learn • MLflow • Deploy

Prof^a Dra. Crishna Irion

Agenda do Encontro

1

Deploy de Modelo

Por que produção é diferente do notebook?

2

Exportando o Modelo

Pickle ou joblib: qual usar?

3

Pipelines no Scikit-Learn

Como unir pré-processamento e modelo?

4

MLflow

Como registrar e comparar experimentos?

5

Salvar, Abrir e Praticar

Dinâmica e desafio no Colab

PARTE 1

Deploy de Modelo: da Bancada à Produção

Entender o que muda quando um modelo sai do notebook e vai para produção — e por que o pipeline de pré-processamento vai junto.

Por que "funciona no meu notebook" não é suficiente?

Modelo no notebook: 92% de acurácia
Em produção: acurácia cai para 61%
Pré-processamento não foi replicado
Aprovamos esse deploy assim mesmo?

Funciona aqui ≠ funciona lá.

Por que um modelo com ótima métrica no notebook pode falhar em produção?

O pipeline de pré-processamento foi salvo e versionado junto com o modelo?

Quem — e como — vai rodar esse modelo depois que ele sair do notebook?

Dois ambientes em conflito: Treino x Produção

TREINO

(notebook / experimentação)

- dados históricos, já limpos e organizados em um DataFrame
- importa explorar, testar e iterar rápido
- ex: rodar fit_transform em um Jupyter Notebook
- ambiente controlado: você decide cada passo manualmente

PRODUÇÃO

(servidor / usuário real)

- dados novos, chegando um a um, nem sempre limpos
- importa robustez, velocidade e repetibilidade
- ex: aplicar transform (nunca fit) a cada nova requisição
- ambiente automatizado: o pipeline decide por você, sempre igual

O pipeline existe justamente para que treino e produção tratem os dados exatamente da mesma forma.

Como preparar um modelo para o deploy:

3 perguntas

1

Em que formato salvar?

pickle é mais simples; joblib é mais eficiente para objetos com muitos arrays (como modelos de ML)

2

O pipeline vai junto?

salvar só o modelo não basta — o pré-processamento precisa ser exportado junto, como um único objeto

3

Como validar antes de subir?

testar o modelo exportado com dados novos, fora do notebook, antes de considerar o deploy pronto

Não existe deploy "de última hora" — ele começa a ser planejado desde o treino.

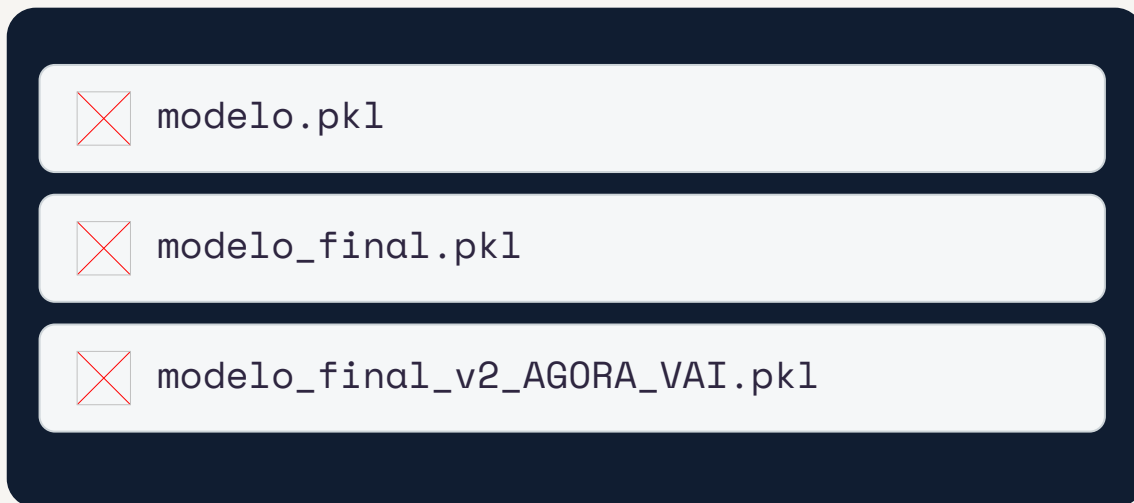
PARTE 2

Exportando o Modelo

Como salvar um modelo treinado em disco — e recarregá-lo depois — usando pickle e joblib.

Modelos salvos sem histórico

Pasta do projeto, três semanas depois:



Perguntas sem resposta

- Qual deles teve a melhor acurácia?
- Com quais hiperparâmetros?
- Treinado com quais dados?
- Com qual pré-processamento?
- Em qual versão do Python e do scikit-learn?

O MLflow guarda a resposta para cada uma dessas perguntas.

Pickle: a forma mais simples de exportar

como funciona

serializa qualquer objeto Python, incluindo o modelo treinado

por que atrapalha

- 1 · não é otimizado para objetos com muitos arrays NumPy (comum em modelos de ML)
- 2 · pode ficar lento e pesado com modelos grandes (ex: Random Forest com muitas árvores)

A alternativa: Joblib

Parte do próprio ecossistema scikit-learn e otimizado para objetos com muitos arrays NumPy — como modelos treinados. Mesma ideia do pickle, só que mais rápido e eficiente para esse caso.

Por isso é a opção recomendada oficialmente pela documentação do scikit-learn.

Ponto de atenção: pickle e joblib salvam bytecode Python — o mesmo modelo pode não abrir em outra versão da biblioteca ou do Python.

Pickle vs Joblib: comparando na prática

PICKLE

"import pickle"

biblioteca padrão do Python — não precisa instalar nada
serve para qualquer objeto Python (não só modelos)
risco: mais lento para arrays NumPy grandes

JOBLIB

"import joblib"

já vem junto com o scikit-learn
muito mais rápido para objetos com arrays grandes
padrão recomendado pela comunidade de ML

Código: exportando um modelo com JOBLIB



1. Salvando o modelo treinado

Passe o objeto treinado e o caminho do arquivo de saída.

```
import joblib
joblib.dump(modelo, 'modelo.joblib')
# salva o modelo treinado em disco
# funciona bem com objetos grandes (arrays NumPy)
```



2. Carregando o modelo salvo

O objeto carregado é idêntico ao original — pronto para prever, sem re-treinar.

```
import joblib
modelo_carregado = joblib.load('modelo.joblib')
modelo_carregado.predict(X_novo)
# nao precisa treinar de novo
# 0.94 -> mesma performance do modelo original
```

joblib.dump salva o modelo em um arquivo; joblib.load recupera o objeto exatamente como estava.

Pipelines no Scikit-Learn

Unir pré-processamento e modelo em um único objeto — para que treino e produção usem exatamente os mesmos passos.

| Por que não basta um `.fit()`?

`fit()` = ?

Pipeline é um objeto que encadeia pré-processamento e modelo em uma única sequência.

1

Sem pipeline

cada etapa (imputar, codificar, escalar) é escrita à mão, fora de ordem, fácil de esquecer um passo

2

Com pipeline

todas as etapas viram um único objeto, treinado e aplicado com `.fit()` e `.predict()`

3

Por que importa

elimina a divergência entre o pré-processamento do treino e o da produção

Pipeline na prática: uma etapa por tipo de coluna

1

Colunas numéricas

SimpleImputer +
StandardScaler —
preenche valores nulos e
coloca na mesma escala

2

Colunas categóricas

SimpleImputer +
OneHotEncoder —
preenche valores nulos e
transforma categorias em
números

3

Texto / outras

transformadores específicos
—
ex: TfidfVectorizer para texto;
cada tipo de dado tem o seu

ColumnTransformer aplica a transformação certa em cada grupo de colunas — dentro de um único Pipeline.

CONCEITO

Pipeline: da entrada bruta ao modelo pronto

Uma única chamada `.fit(X_train, y_train)` roda todas as etapas — imputação, encoding, escala e o modelo — na ordem certa, sempre.

| Pipeline: cada etapa roda automaticamente, na ordem

ENTRADA → **PRONTO**

- imputa valores nulos nas colunas certas
- codifica variáveis categóricas
- padroniza as variáveis numéricas
- treina (ou aplica) o modelo, por último

O Pipeline transforma um processo manual e propenso a erro em uma sequência automática e repetível.

.fit_transform() x .transform(): cuidado com o vazamento de dados

NO TREINO: fit_transform()

aprende os parâmetros da transformação
(ex: média para imputar, escala para normalizar)
e aplica essa transformação nos dados de treino
usado só uma vez, nos dados de treino
aqui é onde o pipeline "aprende" o pré-processamento

EM PRODUÇÃO: transform()

aplica a transformação já aprendida, sem recalculá-la
usado em teste, validação e em cada nova predição
nunca chama fit de novo — senão o modelo "vê" dados que não devia
é assim que o pipeline evita vazamento de dados (data leakage)

Regra de ouro: fit_transform só no treino; transform em todo o resto.

Código: Pipeline + ColumnTransformer no Scikit-Learn

1. Definir o ColumnTransformer

Agrupar a transformação certa para cada tipo de coluna.

```
from sklearn.compose import
ColumnTransformer
ct = ColumnTransformer([('num',
StandardScaler(), num_cols), ('cat',
OneHotEncoder(), cat_cols)])
```

2. Montar e usar o Pipeline

Une o pré-processamento e o modelo em um único objeto treinável.

```
from sklearn.pipeline import Pipeline
pipeline = Pipeline([
    ('preprocessamento', ct),

    ('modelo', RandomForestClassifier())
])

# treina tudo de uma vez:
pipeline.fit(X_train, y_train)

pipeline.predict(X_novo)
```

Código: exportando o Pipeline completo



1. Salvar o pipeline treinado

joblib.dump funciona com qualquer objeto — inclusive um Pipeline inteiro.

```
import joblib
joblib.dump(pipeline, 'pipeline_completo.joblib')
# pré-processamento + modelo, em um so arquivo
```



2. Usar em produção

Um único .predict() já aplica pré-processamento e modelo, na ordem certa.

```
import joblib
pipeline = joblib.load('pipeline_completo.joblib')
pipeline.predict(dados_novos_brutos)
# dados_novos_brutos: sem nenhum pre-processamento manual
# o pipeline cuida de tudo sozinho

# resultado identico ao pipeline original
```

Salvar o Pipeline inteiro garante que o pré-processamento vai junto com o modelo para a produção.

PARTE 4

MLflow: Rastreamento de Experimentos

Registrar parâmetros, métricas e artefatos de cada experimento — para nunca mais perder de vista qual configuração deu qual resultado.

O que é MLflow

1

Tracking

Registra parâmetros, métricas, artefatos e tags de cada execução.

2

Models

Formato padrão para salvar o modelo junto com o seu ambiente.

3

Model Registry

Versiona modelos e controla a promoção do teste à produção.

Plataforma open source para gerenciar o ciclo de vida de modelos de machine learning, qualquer que seja a biblioteca usada para treiná-los.

Por que rastrear experimentos?

o problema

treinar vários modelos com configurações diferentes e perder o controle de qual gerou qual resultado

por que atrapalha

- 1 · sem registro, é fácil reexecutar por engano a configuração errada
- 2 · comparar experimentos "de cabeça" não escala com muitos testes

O que considerar

O **MLflow** registra automaticamente parâmetros, métricas e artefatos de cada execução (run). Cada run fica salvo, comparável e reproduzível depois.

Assim, qualquer pessoa do time pode ver o que já foi tentado.

Ponto de atenção: o MLflow é abrangente — comece pelo tracking (rastreamento) antes de explorar registro e deploy de modelos.

O vocabulário do MLflow

Termo	O que é	Exemplo nos notebooks
Experimento	Agrupa as execuções de um mesmo problema	Aula12 - Iris (multiclasse)
Run	Uma execução do código	setup + compare_models + finalize
Parâmetro	Entrada, configuração do experimento	session_id=123, max_depth=5
Métrica	Resultado numérico	cv_Accuracy, holdout_R2
Artefato	Qualquer arquivo anexado ao run	ranking_modelos.csv, .png
Tag	Rótulo livre para filtrar runs	melhor_modelo=LogisticRegression
Modelo	Pipeline salvo no formato MLflow	model.pkl + MLmodel

Dois jeitos de integrar

automático

cada modelo do
compare_models vira um
run

como fazer

- 1 · `setup(data, target='species', log_experiment=True, log_plots=True)`
- 2 · uma linha resolve tudo, mas com menos controle sobre o conteúdo

Manual · usado nos notebooks

Você decide o que entra em cada run: `mlflow.start_run() + log_params + log_metric + log_model`. — como modelos treinados. Mesma ideia do pickle, só que mais rápido e eficiente para esse caso.

Controle total e fácil de explicar — mais linhas de código.

Escolha um dos dois: usar ambos ao mesmo tempo cria runs duplicados.

O que o MLflow registra em cada execução

1

Parâmetros

os hiperparâmetros
usados nessa execução
(ex: n_estimators,
max_depth)

2

Métricas

os resultados obtidos
(ex: acurácia, F1-score)
— para comparar
execuções

3

Artefatos

arquivos gerados, como
o próprio modelo salvo
ou gráficos

MLflow não escolhe o melhor modelo por você — mas registra tudo o que você precisa para decidir.

Código: MLflow — tracking básico



1. Iniciar um run e registrar parâmetros

Cada chamada a `mlflow.start_run()` cria uma nova execução rastreável.

```
import mlflow
with mlflow.start_run():
    mlflow.log_param('n_estimators', 100)
    mlflow.log_param('max_depth', 5)
    modelo.fit(X_train, y_train)
```



2. Registrar métricas e o modelo

Depois, é só abrir a interface do MLflow (`mlflow ui`) para comparar todas as execuções.

```
mlflow.log_metric('acuracia', 0.94)
mlflow.sklearn.log_model(modelo, 'modelo')
```

`mlflow.start_run()` abre um registro; `log_param` e `log_metric` salvam o que importa dentro dele.

PARTE 5

Mão na Massa

Dinâmica (offline): Monte a Esteira do Deploy

COMO FUNCIONA (grupos de 3–4 pessoas)

- cada grupo recebe um envelope com cartões de etapas embaralhados (imputar, codificar, escalar, treinar, salvar, servir)
- alguns cartões são "pegadinhas" — passos fora de ordem ou repetidos
- o grupo tem 8 minutos para montar a esteira na ordem correta
- ao final, cada grupo defende sua ordem para a turma

REGRAS DO JOGO (o que vale pontos)

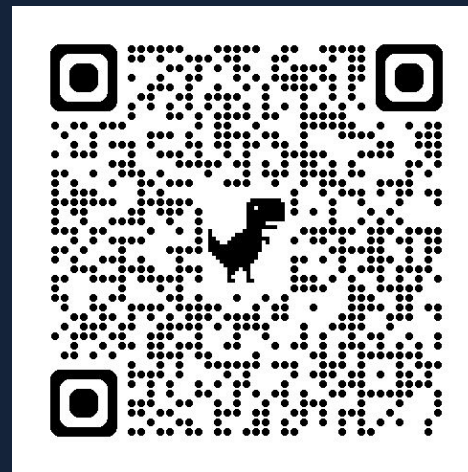
- ordem certa: pré-processamento sempre antes do treino, nunca depois
- identificar a pegadinha: um cartão de "fit_transform em produção" escondido no meio
- o grupo mais rápido com a ordem certa ganha o direito de rodar o código antes
- objetivo real: sentir na mão por que a ordem e a consistência do pipeline importam

A esteira errada em papel custa risadas. A esteira errada em produção custa dados incorretos para usuários reais.

PARTE 5

Mão na Massa

Código com desafio



<https://encurtador.com.br/gXTS>

Colab



1. Como usar

Rode depois de tentar sozinho — compare sua solução com a nossa antes de perguntar "está certo?".

```
# pipeline completo, pronto para rodar
# + explicacao de cada etapa do ColumnTransformer
# + por que fit_transform so aparece uma vez
# + checklist de erros comuns
```



2. O que tem a mais

Comentários explicando o porquê de cada escolha, não só o "como".

```
# tracking do MLflow ja configurado
# print com o caminho do modelo salvo
# celula extra: como abrir o mlflow ui
# dica final:
# compare o resultado com o do seu grupo na dinamica
```

Dúvidas? Vamos conversar.

Profª Dra. Crishna Irion

crishna.irion@prof.inteli.edu.br

